

Rapport J10

SQL WHERE, GROUP BY, HAVING et filtrage analytique

1. Identification

- Date : 23 septembre 2026
- Semaine : S02 - Jour : J10
- Base : sql_training
- Table de travail : raw.training_events
- Durée prévue : 90 min
- État du timer communiqué : 53 min effectuées, 37 min restantes
- Durée réelle définitive : à compléter à la clôture de J10
- Écart prévu/réel définitif : à compléter à la clôture de J10

2. Objectifs du jour

- Comprendre précisément la différence entre WHERE et HAVING.
- Comprendre l'ordre logique WHERE -> GROUP BY -> agrégations -> HAVING.
- Combiner filtres de lignes et filtres de groupes.
- Consolider COUNT, AVG, ROUND, MAX, GROUP BY et ORDER BY.
- Diagnostiquer l'erreur produite par une fonction d'agrégation placée dans WHERE.
- Réaliser un challenge autonome et valider un script SQL reproductible avec psql -f.

3. Validation de l'environnement et du dataset

- Le dataset de départ contient 15 lignes, 4 source_system distincts et 6 event_type distincts.
- Les identifiants observés vont de MIN(event_id)=1 à MAX(event_id)=18.
- Les quatre sources présentes sont manual_psql, microgrid_poc, sensor_gateway et tomra_training.
- Capture : J10-01_environment-dataset-validation.png

4. Notion centrale - WHERE vs HAVING

- WHERE filtre les lignes individuelles avant la formation des groupes.
- HAVING filtre les groupes après GROUP BY, lorsque les agrégats tels que COUNT(*) sont disponibles.
- Ordre conceptuel retenu : FROM -> WHERE -> GROUP BY -> agrégations -> HAVING -> SELECT -> ORDER BY.

5. Exercice - WHERE avant agrégation

```
SELECT
    source_system,
    COUNT(*) AS event_count,
    ROUND(AVG(event_value), 2) AS avg_value
FROM raw.training_events
WHERE event_value >= 20
GROUP BY source_system
ORDER BY event_count DESC, source_system;
```

- Résultat : 3 groupes. microgrid_poc = 5 événements / moyenne 261.99 ; sensor_gateway = 4 / 28.00 ; manual_psql = 1 / 125.50.
- tomra_training disparaît parce que toutes ses lignes ont event_value < 20 et sont éliminées avant GROUP BY.
- Capture : J10-02_where-before-aggregation.png

6. Exercice - HAVING sur les groupes

```
SELECT
    source_system,
    COUNT(*) AS event_count,
    ROUND(AVG(event_value), 2) AS avg_value
FROM raw.training_events
GROUP BY source_system
HAVING COUNT(*) >= 3
ORDER BY event_count DESC, source_system;
```

- Résultat : microgrid_poc = 7 ; sensor_gateway = 4 ; tomra_training = 3. manual_psql est éliminé car son groupe ne contient qu'une ligne.
- Conclusion formulée pendant la séance : HAVING intervient après la formation des groupes, contrairement à WHERE qui filtre les lignes avant.
- Capture : J10-03_having-group-filter.png

7. Erreur reproduite, diagnostic et résolution

```
SELECT
    source_system,
    COUNT(*) AS event_count
FROM raw.training_events
WHERE COUNT(*) >= 3
GROUP BY source_system;
```

- Erreur PostgreSQL observée : ERROR: aggregate functions are not allowed in WHERE.
- Cause : COUNT(*) dépend du regroupement, alors que WHERE est évalué avant GROUP BY.
- Résolution : déplacer le prédicat d'agrégation dans HAVING COUNT(*) >= 3.

```
SELECT
    source_system,
    COUNT(*) AS event_count
FROM raw.training_events
GROUP BY source_system
HAVING COUNT(*) >= 3
ORDER BY event_count DESC;
```

- Capture erreur : J10-04_aggregate-in-where-error.png
- Capture correction : J10-05_where-having-error-correction.png

8. Combinaison WHERE + GROUP BY + HAVING

```
SELECT
    source_system,
    COUNT(*) AS event_count,
    ROUND(AVG(event_value), 2) AS avg_value,
    MAX(event_value) AS max_value
FROM raw.training_events
WHERE event_value >= 10
GROUP BY source_system
HAVING COUNT(*) >= 2
ORDER BY avg_value DESC;
```

- Résultat : 2 groupes - microgrid_poc (7, moyenne 190.99, max 315.90) et sensor_gateway (4, moyenne 28.00, max 31.40).
- tomra_training est supprimé par WHERE car ses valeurs sont < 10.
- manual_psql passe WHERE avec 125.50, mais son groupe ne compte qu'une ligne et est supprimé par HAVING COUNT(*) >= 2.
- Capture : J10-06_where-groupby-having-combination.png

9. Challenge autonome

```
SELECT
    event_type,
    COUNT(*) AS event_count,
    ROUND(AVG(event_value), 2) AS avg_value,
    MAX(event_value) AS max_value
FROM raw.training_events
WHERE event_value >= 10
GROUP BY event_type
HAVING COUNT(*) >= 2
ORDER BY event_count DESC, event_type;
```

- Préviation avant exécution : 4 groupes.
- Résultat confirmé : temperature (4, 28.00, 31.40), voltage (3, 230.27, 231.80), current (2, 13.48, 14.20), sensor_reading (2, 309.58, 315.90).
- machine_alarm est éliminé par WHERE. training_test passe WHERE mais est éliminé par HAVING car il ne possède qu'une observation.
- Capture : J10-07_autonomous-where-having-challenge.png

10. Script reproductible et validation

- Script : documentation/semaine-02/jour-10/code/J10_where_having_aggregations.sql


```
psql -U postgres -d sql_training -f ".\documentation\semaine-02\jour-10\code\J10_where_having_aggregations.sql"
```
- Le script s'est exécuté de bout en bout. La requête volontairement incorrecte est conservée en commentaire pour ne pas interrompre l'exécution automatisée.

- Contrôle final : total_rows=15, min_event_id=1, max_event_id=18.
- Aucune donnée n'a été modifiée pendant J10.
- Capture : J10-08_where-having-script-validation.png

11. KPI de progression

- Dataset validé : Oui.
- WHERE avant agrégation : acquis.
- HAVING après agrégation : acquis.
- Combinaison WHERE + GROUP BY + HAVING : réussie.
- Erreur d'agrégat dans WHERE : reproduite, expliquée et corrigée.
- Challenge autonome : réussi.
- Prévion du challenge : 4 groupes prévus / 4 obtenus.
- Script reproductible : Oui.
- Validation psql -f : réussie.
- Modification involontaire des données : aucune.
- Durée prévue : 90 min.
- Temps consommé au moment du rapport : 53 min.
- Temps restant au moment du rapport : 37 min.

12. Validation des acquis

- L'utilisateur sait distinguer un filtre de lignes d'un filtre de groupes.
- L'utilisateur sait choisir WHERE ou HAVING selon le niveau de filtrage.
- L'utilisateur sait expliquer pourquoi COUNT(*) est accepté dans HAVING mais pas dans WHERE.
- L'utilisateur sait combiner filtrage, regroupement, agrégations et tri.
- L'utilisateur sait prévoir le nombre de groupes puis confronter la prévision au résultat.
- L'utilisateur sait transformer les exercices en script SQL réexécutable.

13. Potentiel de réutilisation

- Tutoriel : SQL WHERE vs HAVING avec PostgreSQL - filtrer les lignes et les groupes.
- Vidéo éducative : 5 à 8 min, avec démonstration de l'erreur WHERE COUNT(*) puis correction.
- Module de formation : 20 à 30 min avec dataset, prédiction, erreur guidée et challenge autonome.
- Portfolio : preuve de raisonnement analytique, diagnostic d'erreur, agrégations et reproductibilité.

14. Git et clôture restante

- Les rapports MD/DOCX/PDF doivent être placés dans documentation/semaine-02/jour-10/.
- Vérifier ensuite git status --short, ajouter le dossier J10, inspecter le staging, effectuer le commit et le push.
- Message de commit proposé : docs: complete J10 SQL WHERE HAVING and grouped filtering training
- J10 ne sera considéré terminé qu'après validation Git finale et mise à jour de la durée réelle.

15. Conclusion

- J10 fait évoluer les acquis de J09 sur GROUP BY vers le filtrage analytique des groupes.
- La distinction WHERE/HAVING a été comprise, testée par une erreur volontaire, corrigée, puis réutilisée avec succès dans un challenge autonome.
- Statut technique : validé. Statut documentaire : rapports générés. Statut Git : à finaliser.